

Cesto

Banco de dados

Arquitetura dos seis bancos PostgreSQL, diagramas, tabelas, índices, máquinas de estado, outbox, chaves do Redis e preparo do ambiente.

Copyright (©) 2026 Clara A. Portes

Versão 1.0.0 · Outubro de 2026 · Licença MIT

Sumário

1. Visão geral **3**

Um banco por serviço 3

Como os dados se relacionam entre serviços 3

Redis 3

Convenções 3

2. cesto_auth **5**

Diagrama 5

users 5

oauth_clients 5

authorization_codes 6

refresh_tokens 6

signing_keys 7

3. cesto_catalog **8**

Diagrama 8

categories 8

products 8

outbox_events 9

4. cesto_inventory **10**

Diagrama 10

stock_items 10

reservations 10

processed_events e outbox_events 11

5. cesto_orders **12**

Diagrama 12

orders 12

order_items 13

order_history 13

6. cesto_payments e cesto_notifications **14**

Diagrama 14

payments 14

notifications 15

7. Máquinas de estado **16**
Pedido 16
Reserva de estoque 16
Refresh token 16
Pagamento 17

8. Eventos, outbox e Redis **18**
Outbox transacional 18
Chaves do Redis 18
Envelope de evento 18

9. Como preparar o banco **19**
Com Docker (recomendado) 19
PostgreSQL próprio 19
Mudar o esquema 19
Backup e retenção 19
Consultas úteis 20

CAPÍTULO 1

Visão geral

Um banco por serviço

O Cesto segue o padrão **database per service**: cada microsserviço é o único dono do seu banco. Isso permite evoluir, escalar e fazer backup de cada parte separadamente, e impede que uma mudança de tabela quebre outro serviço.

Banco	Dono	Conteúdo
`cesto_auth`	auth	Usuários, clientes OAuth2, códigos, refresh tokens, chaves de assinatura
`cesto_catalog`	catalog	Categorias, produtos, outbox
`cesto_inventory`	inventory	Saldos, reservas, outbox, eventos processados
`cesto_orders`	orders	Pedidos, itens, histórico, outbox, eventos processados
`cesto_payments`	payments	Pagamentos, outbox, eventos processados
`cesto_notifications`	notifications	Notificações

Os seis bancos podem morar no mesmo servidor PostgreSQL 16 (como no Docker Compose) ou em servidores diferentes. Basta apontar o `DATABASE_URL` de cada serviço.

Como os dados se relacionam entre serviços

Não existem chaves estrangeiras entre bancos. As referências cruzadas são identificadores (`product_id`, `order_id`, `user_id`) mantidos por eventos:

- `products.id` do catálogo vira `stock_items.product_id` no estoque pelo evento `product.created`.
- `orders.id` é referenciado por `reservations.order_id`, `payments.order_id` e `notifications.order_id`.
- `users.id` do auth é o `sub` do JWT e aparece como `user_id` em pedidos, pagamentos e notificações.
- Os itens do pedido guardam uma cópia do SKU, do nome e do preço no momento da compra.

Redis

O Redis não é fonte da verdade de nenhum dado de negócio, exceto o carrinho, que é descartável por natureza. Ele guarda os fluxos de eventos, o carrinho e os contadores de rate limit.

Convenções

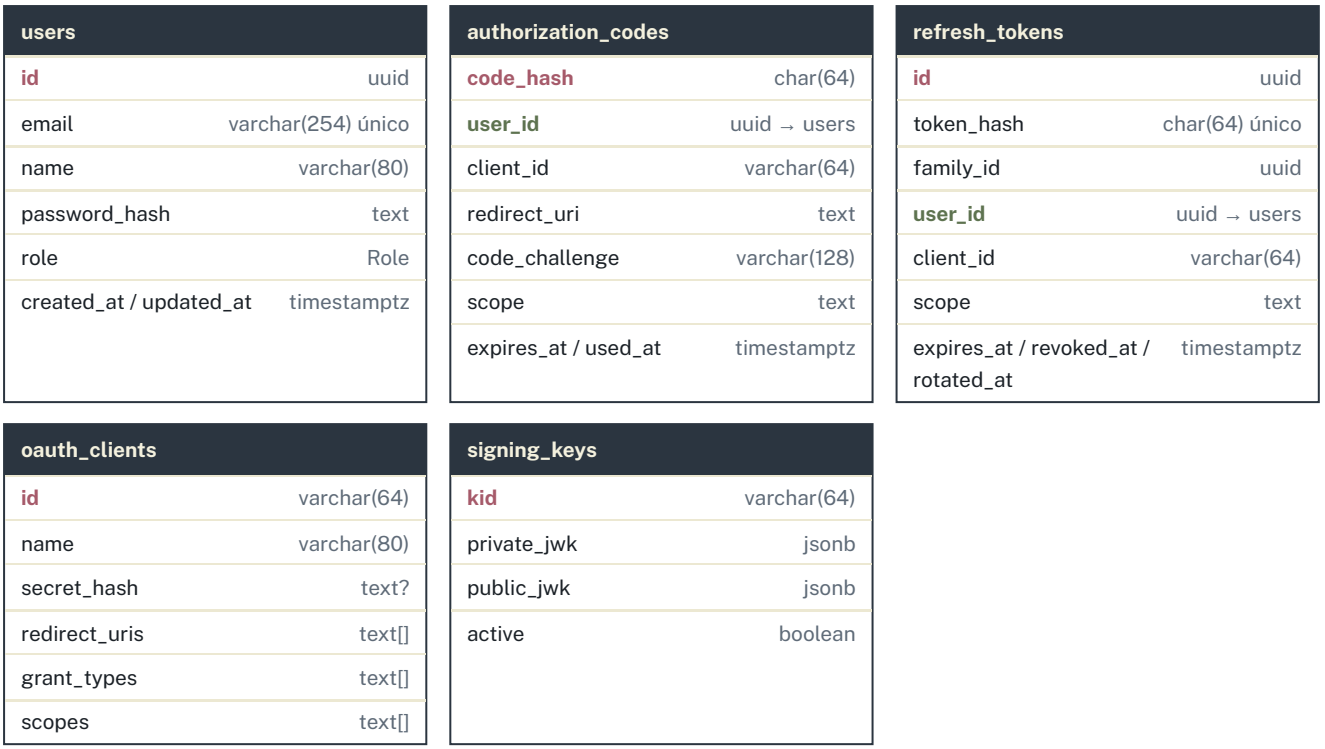
Regra	Exemplo
Tabelas no plural, em snake_case	`order_items`
Colunas em snake_case; no código, camelCase via `@map`	`price_cents` ↔ `priceCents`
Chaves primárias UUID v4	`id UUID`

Regra	Exemplo
Dinheiro em centavos inteiros	`total_cents INTEGER`
Datas com fuso e milissegundos	`TIMESTAMPTZ(3)`
Enums nativos do PostgreSQL	`"OrderStatus"`
Restrições de integridade no banco	`CHECK (available >= 0)`
Índices nomeados como o Prisma	`orders_user_id_created_at_idx`

CAPÍTULO 2

cesto_auth

Diagrama



users

Coluna	Tipo	Nulo	Padrão	Observação
id	UUID	não	uuid()	PK
email	VARCHAR(254)	não	—	Único (`users_email_key`), sempre minúsculo
name	VARCHAR(80)	não	—	
password_hash	TEXT	não	—	`scrypt$N$sal$hash`
role	"Role"	não	CUSTOMER	CUSTOMER ou ADMIN
created_at	TIMESTAMPTZ(3)	não	now()	
updated_at	TIMESTAMPTZ(3)	não	—	Atualizado pelo Prisma

oauth_clients

Coluna	Tipo	Nulo	Padrão	Observação
id	VARCHAR(64)	não	—	PK; ex.: `cesto-web`, `cesto-internal`
name	VARCHAR(80)	não	—	

Coluna	Tipo	Nulo	Padrão	Observação
secret_hash	TEXT	sim	—	Nulo = cliente público (exige PKCE)
redirect_uris	TEXT[]	—	—	Endereços de retorno permitidos
grant_types	TEXT[]	—	—	authorization_code, refresh_token, client_credentials
scopes	TEXT[]	—	—	Teto de escopos do cliente
created_at	TIMESTAMPTZ(3)	não	now()	

authorization_codes

Coluna	Tipo	Nulo	Padrão	Observação
code_hash	CHAR(64)	não	—	PK; SHA-256 do código
client_id	VARCHAR(64)	não	—	
user_id	UUID	não	—	FK users, ON DELETE CASCADE
redirect_uri	TEXT	não	—	Precisa bater na troca
code_challenge	VARCHAR(128)	não	—	Desafio PKCE S256
scope	TEXT	não	—	Escopos concedidos
expires_at	TIMESTAMPTZ(3)	não	—	Índice; 60 s por padrão
used_at	TIMESTAMPTZ(3)	sim	—	Uso único
created_at	TIMESTAMPTZ(3)	não	now()	

refresh_tokens

Coluna	Tipo	Nulo	Padrão	Observação
id	UUID	não	uuid()	PK
token_hash	CHAR(64)	não	—	Único; SHA-256 do token
family_id	UUID	não	—	Índice; une as rotações de um mesmo login
client_id	VARCHAR(64)	não	—	
user_id	UUID	não	—	FK users, índice
scope	TEXT	não	—	
expires_at	TIMESTAMPTZ(3)	não	—	30 dias por padrão
revoked_at	TIMESTAMPTZ(3)	sim	—	Logout ou reuso detectado
rotated_at	TIMESTAMPTZ(3)	sim	—	Preenchido quando trocado por um novo
created_at	TIMESTAMPTZ(3)	não	now()	

signing_keys

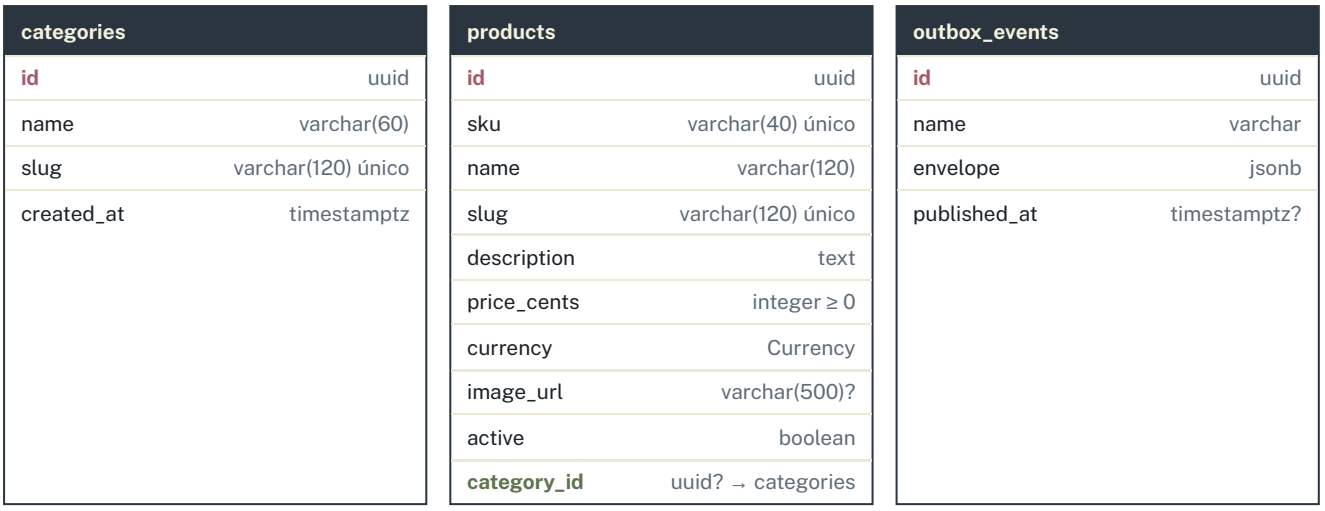
Coluna	Tipo	Nulo	Padrão	Observação
kid	VARCHAR(64)	não	—	PK; vai no cabeçalho do JWT
private_jwk	JSONB	não	—	Chave privada RSA (proteja o backup)
public_jwk	JSONB	não	—	Publicada no JWKS
active	BOOLEAN	não	true	Chave usada para assinar
created_at	TIMESTAMPTZ(3)	não	now()	

> A tabela `signing_keys` contém material criptográfico. Restrinja o acesso ao banco `cesto_auth` e criptografe os backups.

CAPÍTULO 3

cesto_catalog

Diagrama



categories

Coluna	Tipo	Nulo	Padrão	Observação
id	UUID	não	uuid()	PK
name	VARCHAR(60)	não	—	
slug	VARCHAR(120)	não	—	Único; usado no filtro `?category=`
created_at	TIMESTAMPTZ(3)	não	now()	

products

Coluna	Tipo	Nulo	Padrão	Observação
id	UUID	não	uuid()	PK
sku	VARCHAR(40)	não	—	Único; maiúsculas, números e hífen
name	VARCHAR(120)	não	—	Busca sem diferenciar maiúsculas
slug	VARCHAR(120)	não	—	Único
description	TEXT	não	''	Busca
price_cents	INTEGER	não	—	`CHECK (price_cents >= 0)`
currency	"Currency"	não	BRL	BRL, USD ou EUR
image_url	VARCHAR(500)	sim	—	
active	BOOLEAN	não	true	Desativar em vez de apagar

Coluna	Tipo	Nulo	Padrão	Observação
category_id	UUID	sim	—	FK categories, ON DELETE SET NULL, índice
created_at / updated_at	TIMESTAMPTZ(3)	não	now() / —	

Índice composto `(active, created_at)` atende a listagem padrão da loja, que mostra só produtos ativos, dos mais novos para os mais antigos.

outbox_events

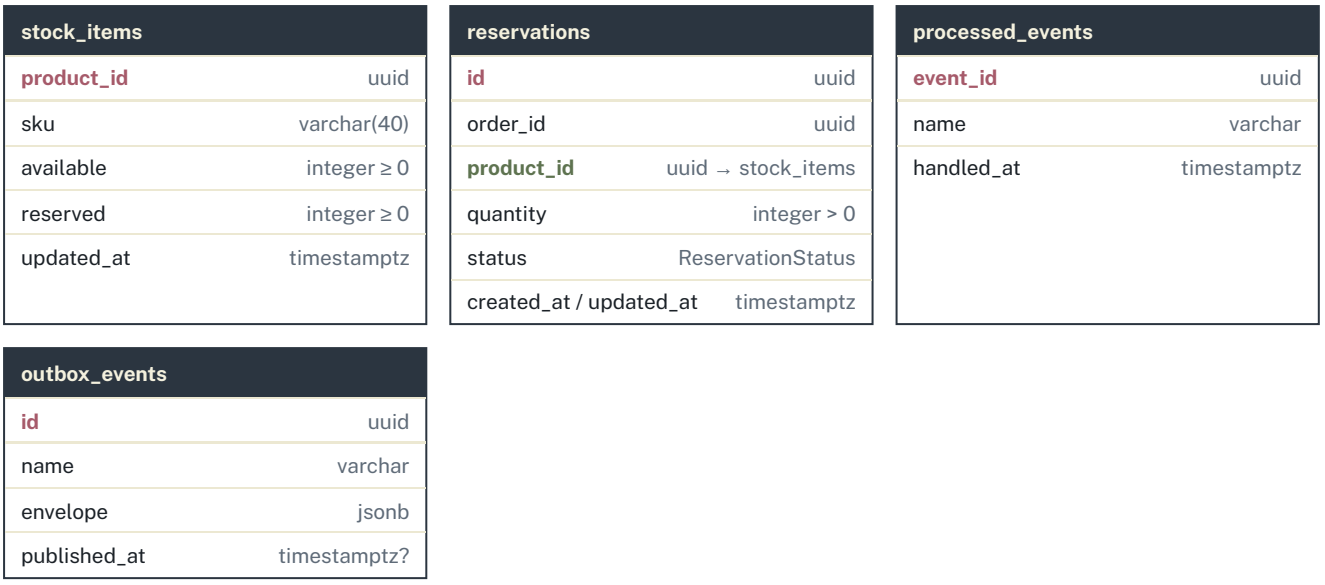
Coluna	Tipo	Nulo	Padrão	Observação
id	UUID	não	—	PK; mesmo id do envelope do evento
name	VARCHAR(60)	não	—	Nome do evento
envelope	JSONB	não	—	Envelope completo, validado com Zod
created_at	TIMESTAMPTZ(3)	não	now()	
published_at	TIMESTAMPTZ(3)	sim	—	Preenchido pelo relay

Índice `(published_at, created_at)` para o relay encontrar rapidamente o que falta publicar, na ordem.

CAPÍTULO 4

cesto_inventory

Diagrama



stock_items

Coluna	Tipo	Nulo	Padrão	Observação
product_id	UUID	não	—	PK; mesmo id do catálogo
sku	VARCHAR(40)	não	—	Cópia para leitura
available	INTEGER	não	0	`CHECK (available >= 0)`; pode ser vendido
reserved	INTEGER	não	0	`CHECK (reserved >= 0)`; preso a pedidos em andamento
updated_at	TIMESTAMPTZ(3)	não	—	

reservations

Coluna	Tipo	Nulo	Padrão	Observação
id	UUID	não	uuid()	PK
order_id	UUID	não	—	Único com product_id
product_id	UUID	não	—	FK stock_items, ON DELETE RESTRICT
quantity	INTEGER	não	—	`CHECK (quantity > 0)`
status	"ReservationStatus"	não	HELD	HELD, COMMITTED ou RELEASED
created_at / updated_at	TIMESTAMPTZ(3)	não	now() / —	Índice (status, created_at)

processed_events e outbox_events

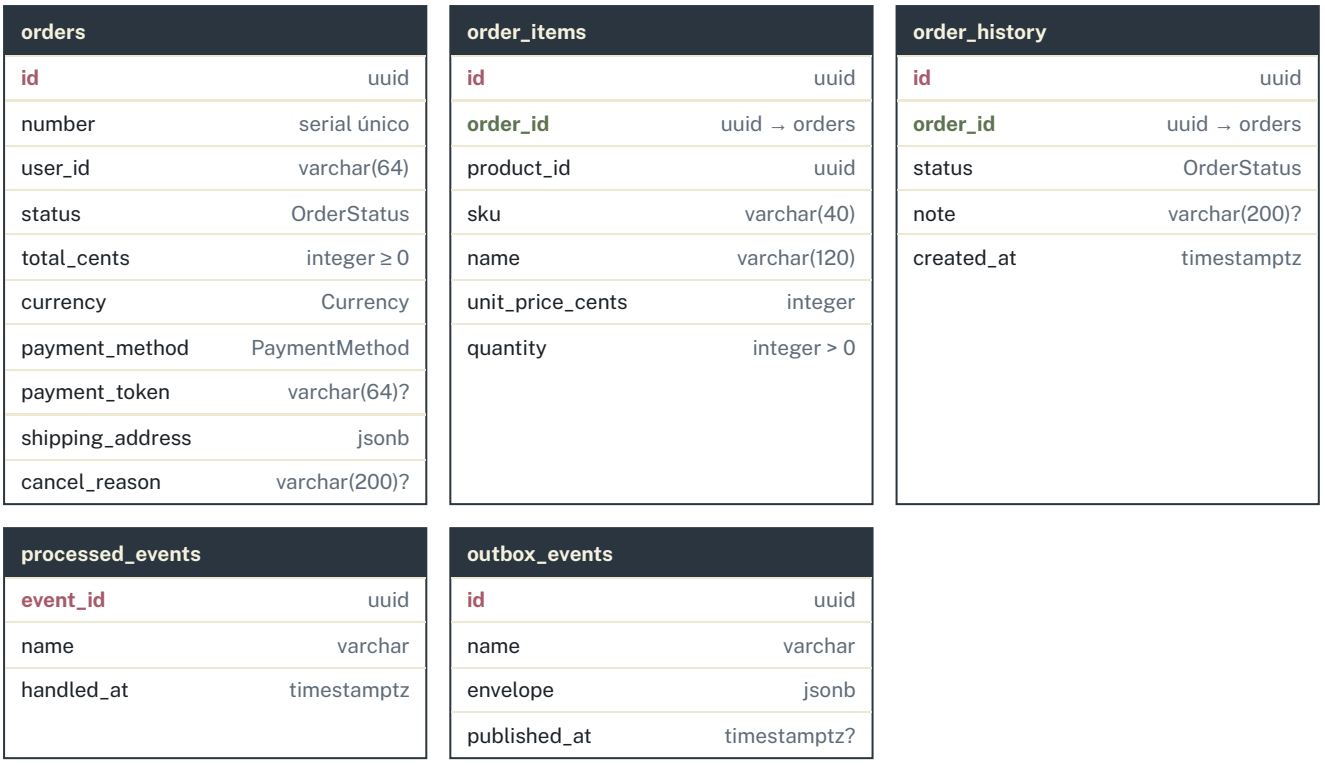
Coluna	Tipo	Nulo	Padrão	Observação
event_id	UUID	não	—	PK; id do evento consumido
name	VARCHAR(60)	não	—	
handled_at	TIMESTAMPTZ(3)	não	now()	

Coluna	Tipo	Nulo	Padrão	Observação
id	UUID	não	—	PK; mesmo id do envelope do evento
name	VARCHAR(60)	não	—	Nome do evento
envelope	JSONB	não	—	Envelope completo, validado com Zod
created_at	TIMESTAMPTZ(3)	não	now()	
published_at	TIMESTAMPTZ(3)	sim	—	Preenchido pelo relay

CAPÍTULO 5

cesto_orders

Diagrama



orders

Coluna	Tipo	Nulo	Padrão	Observação
id	UUID	não	uuid()	PK
number	SERIAL	não	sequência	Único; número amigável (#1, #2…)
user_id	VARCHAR(64)	não	—	`sub` do JWT; índice (user_id, created_at)
status	"OrderStatus"	não	PENDING	Índice (status, created_at)
total_cents	INTEGER	não	—	`CHECK (total_cents >= 0)`
currency	"Currency"	não	BRL	
payment_method	"PaymentMethod"	não	—	card, pix ou boleto
payment_token	VARCHAR(64)	sim	—	Token do provedor (nunca o cartão)
shipping_address	JSONB	não	—	Endereço validado pelo esquema `Address`
cancel_reason	VARCHAR(200)	sim	—	
created_at / updated_at	TIMESTAMPTZ(3)	não	now() / —	

order_items

Coluna	Tipo	Nulo	Padrão	Observação
id	UUID	não	uuid()	PK
order_id	UUID	não	—	FK orders, ON DELETE CASCADE, índice
product_id	UUID	não	—	Referência lógica ao catálogo
sku / name	VARCHAR	não	—	Cópia no momento da compra
unit_price_cents	INTEGER	não	—	Preço congelado
quantity	INTEGER	não	—	`CHECK (quantity > 0)`

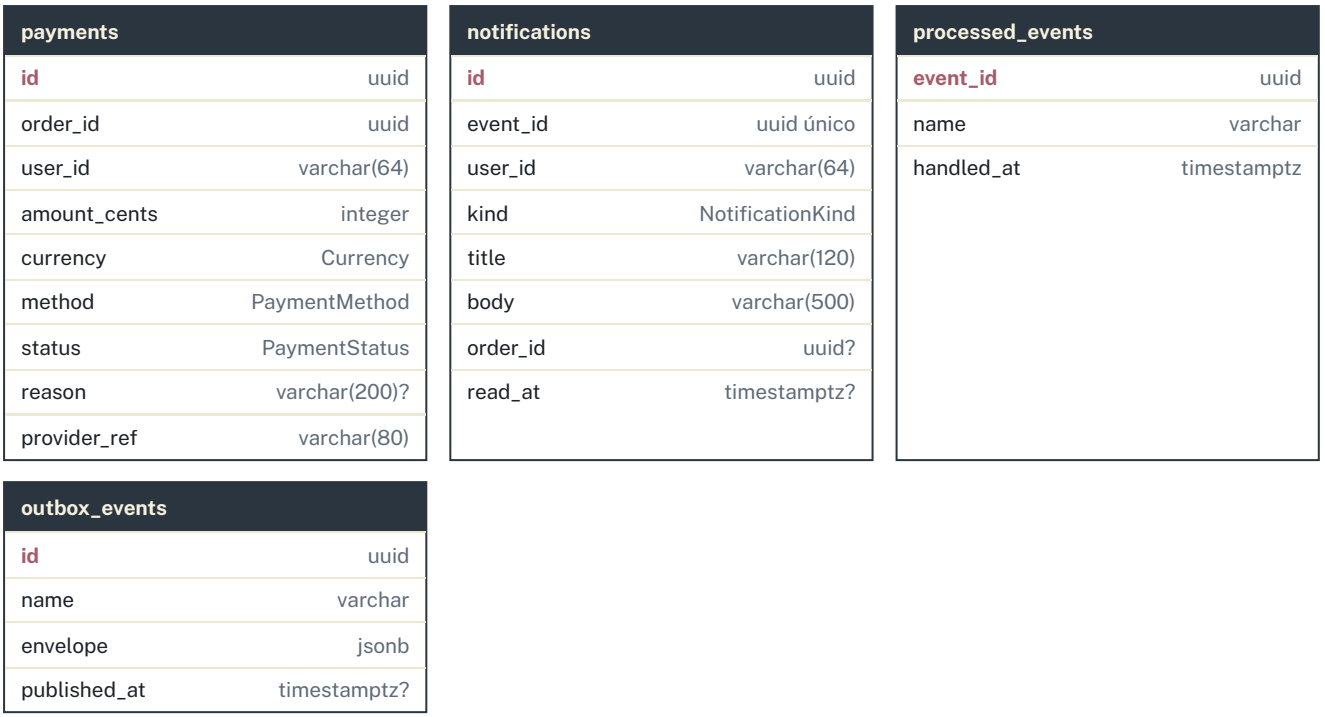
order_history

Coluna	Tipo	Nulo	Padrão	Observação
id	UUID	não	uuid()	PK
order_id	UUID	não	—	FK orders, ON DELETE CASCADE; índice (order_id, created_at)
status	"OrderStatus"	não	—	Status atingido
note	VARCHAR(200)	sim	—	Motivo ou observação
created_at	TIMESTAMPTZ(3)	não	now()	

CAPÍTULO 6

cesto_payments e cesto_notifications

Diagrama



payments

Coluna	Tipo	Nulo	Padrão	Observação
id	UUID	não	uuid()	PK
order_id	UUID	não	—	Índice
user_id	VARCHAR(64)	não	—	Índice (user_id, created_at)
amount_cents	INTEGER	não	—	
currency	"Currency"	não	BRL	
method	"PaymentMethod"	não	—	
status	"PaymentStatus"	não	—	APPROVED, DECLINED ou REFUNDED
reason	VARCHAR(200)	sim	—	Motivo da recusa ou do estorno
provider_ref	VARCHAR(80)	não	—	Referência no provedor
created_at / updated_at	TIMESTAMPTZ(3)	não	now() / —	

notifications

Coluna	Tipo	Nulo	Padrão	Observação
id	UUID	não	uuid()	PK
event_id	UUID	não	—	Único; impede notificação duplicada
user_id	VARCHAR(64)	não	—	Índices (user_id, created_at) e (user_id, read_at)
kind	"NotificationKind"	não	—	WELCOME, ORDER_PLACED, ORDER_CONFIRMED, ORDER_CANCELLED, ORDER_SHIPPED, ORDER_DELIVERED, PAYMENT_DECLINED
title	VARCHAR(120)	não	—	
body	VARCHAR(500)	não	—	
order_id	UUID	sim	—	
read_at	TIMESTAMPTZ(3)	sim	—	Nulo = não lida
created_at	TIMESTAMPTZ(3)	não	now()	

CAPÍTULO 7

Máquinas de estado

Pedido

PENDING → **AWAITING_PAYMENT** → **CONFIRMED** → **SHIPPED** → **DELIVERED**

De	Para	Gatilho
PENDING	AWAITING_PAYMENT	`inventory.reserved`
PENDING	CANCELLED	`inventory.rejected` ou cliente
AWAITING_PAYMENT	CONFIRMED	`payment.approved`
AWAITING_PAYMENT	CANCELLED	`payment.declined` ou cliente
CONFIRMED	SHIPPED	Administrador
CONFIRMED	CANCELLED	Administrador
SHIPPED	DELIVERED	Administrador

A troca de status usa `UPDATE ... WHERE id = ? AND status = ?`. Se duas mensagens tentarem mudar o mesmo pedido ao mesmo tempo, só uma vence, e cada mudança gera uma linha em `order_history`.

Reserva de estoque

Status	Significado	Efeito no saldo
HELD	Pedido em andamento	`available -= q`, `reserved += q`
COMMITTED	Pedido confirmado	`reserved -= q` (baixa definitiva)
RELEASED	Pedido cancelado	HELD: `reserved -= q`, `available += q`; COMMITTED: `available += q`

Refresh token

Situação	Colunas	Resultado
Ativo	rotated_at e revoked_at nulos	Pode ser trocado uma vez
Rotacionado	rotated_at preenchido	Reuso revoga toda a família
Revogado	revoked_at preenchido	Logout, reuso ou código reaproveitado
Expirado	expires_at no passado	Apagado pela limpeza horária

Pagamento

Status	Quando
APPROVED	Cobrança aceita pelo provedor
DECLINED	Cobrança recusada; o motivo fica em `reason`
REFUNDED	Pedido cancelado depois da aprovação

CAPÍTULO 8

Eventos, outbox e Redis

Outbox transacional

1. A mudança de negócio e a linha em `outbox_events` são gravadas na mesma transação.
2. O relay de cada serviço lê até 100 linhas com `published_at IS NULL`, em ordem de criação, publica com `XADD` e marca `published_at`.
3. Se o processo cair entre publicar e marcar, a linha é publicada de novo; quem consome descarta pela tabela `processed_events`.

```
SELECT name, count(*) FROM outbox_events WHERE published_at IS NULL GROUP BY name;
DELETE FROM outbox_events WHERE published_at < now() - interval '30 days';
```

Chaves do Redis

Chave	Tipo	Dono	Conteúdo
`cesto:events:<evento>`	Stream	quem publica	Envelope JSON no campo `event`; limite aproximado de 100 mil entradas
`cesto:events:dead-letter`	Stream	todos	Eventos inválidos ou que falharam 5 vezes, com stream, id, grupo e motivo
`cesto:cart:<userId>`	Hash	cart	productId → quantidade; expira em 30 dias sem uso
`cesto:ratelimit:*`	String	gateway	Contadores por IP

Cada serviço consumidor tem um grupo com o próprio nome (`inventory`, `orders`, `payments`, `notifications`, `cart`) em cada stream que lê.

Envelope de evento

```
{
  "id": "b0c5...", "name": "order.placed", "version": 1,
  "occurredAt": "2026-10-22T16:57:02.311Z", "correlationId": "req-...",
  "payload": { "orderId": "...", "userId": "...", "number": 4, "items": [ ... ],
               "totalCents": 36470, "currency": "BRL", "paymentMethod": "pix" }
}
```

CAPÍTULO 9

Como preparar o banco

Com Docker (recomendado)

O `docker-compose.yml` sobe o PostgreSQL 16 e executa `docker/postgres/init.sql` na primeira vez, criando os seis bancos. Cada serviço roda `prisma migrate deploy` ao iniciar, com até 10 tentativas enquanto o banco sobe. Não há passo manual.

PostgreSQL próprio

```
CREATE ROLE cesto LOGIN PASSWORD 'troque-esta-senha';
CREATE DATABASE cesto_auth OWNER cesto;
CREATE DATABASE cesto_catalog OWNER cesto;
CREATE DATABASE cesto_inventory OWNER cesto;
CREATE DATABASE cesto_orders OWNER cesto;
CREATE DATABASE cesto_payments OWNER cesto;
CREATE DATABASE cesto_notifications OWNER cesto;
```

Depois, com `DB_USER`, `DB_PASSWORD`, `DB_HOST` e `DB_PORT` no `.env`:

```
npm ci
npm run generate
npm run migrate
```

Em servidores separados, defina o `DATABASE_URL` de cada serviço diretamente no ambiente.

Mudar o esquema

1. Edite `services/<serviço>/prisma/schema.prisma`.
2. Na pasta do serviço, rode `npx prisma migrate dev --name descricao` apontando para um banco de desenvolvimento.
3. Revise o SQL gerado em `prisma/migrations/` e acrescente restrições `CHECK` quando fizer sentido.
4. Rode `npm run generate` e `npm run typecheck`: o TypeScript aponta tudo o que precisa mudar.
5. Em produção, `prisma migrate deploy` aplica as migrações novas ao iniciar o serviço.

Backup e retenção

Item	Recomendação
Bancos	`pg_dump -Fc` diário de cada banco; teste a restauração
cesto_auth	Backup criptografado (contém chaves de assinatura e hashes)
outbox_events / processed_events	Apague registros com mais de 30 dias
notifications	Mantenha conforme a política da loja
Redis	AOF ligado no compose; o carrinho é o único dado de negócio

```
for db in auth catalog inventory orders payments notifications; do
  docker compose exec -T postgres pg_dump -U cesto -Fc cesto_$db > backup-$db.dump
done
```

Consultas úteis

```
SELECT status, count(*), sum(total_cents) / 100.0 AS total FROM orders GROUP BY status;

SELECT o.number, h.status, h.note, h.created_at
FROM order_history h JOIN orders o ON o.id = h.order_id
WHERE o.number = 4 ORDER BY h.created_at;

SELECT sku, available, reserved FROM stock_items WHERE available < 5 ORDER BY available;

SELECT r.order_id, r.status, s.sku, r.quantity
FROM reservations r JOIN stock_items s USING (product_id)
WHERE r.status = 'HELD' AND r.created_at < now() - interval '15 minutes';
```